

HC-91 Emulator — Architecture

A source-code-level tour with SysML-style diagrams

Derived from a source audit of `cosmindxu/hc91emu`

This document is the companion to `docs/ARCHITECTURE.md`. It re-states the architecture of `hc91emu`—a cycle-accurate, dependency-free emulator for the I.C.E. Felix HC-91 (a Romanian ZX Spectrum 48K clone) and the wider HC family—and illustrates it with SysML-style diagrams: Block Definition (BDD), Internal Block (IBD), Activity, State-Machine and Sequence diagrams. File/line references are relative to the repository root.

Contents

1	What is being emulated	1
2	Repository layout	2
3	Layered architecture (Block Definition Diagram)	2
4	Machine internal structure (Internal Block Diagram)	3
5	The Z80 CPU core (<code>src/z80.c</code>)	3
5.1	Register model	3
5.2	Incremental T-state accounting	3
5.3	Instruction decode	4
6	The Machine layer (<code>src/machine.c</code>)	4
6.1	Memory maps	4
6.2	Contention model	4
6.3	I/O bus	4
7	The frame loop (Activity Diagram)	5
8	Beam-accurate video (<code>src/video.c</code>)	5
9	Tape subsystem (<code>src/tape.c</code>)	5
9.1	Tape player state machine	6
10	Disk & CP/M (<code>src/fdc.c</code>) — HC-2000	7
11	State: snapshots & RZX	7
12	Input, debugger, frontend, CLI	7
13	Testing & validation	8
14	Notable engineering decisions	8

1 What is being emulated

Aspect	Emulated target
CPU	Z80A / Romanian MMN80 clone @ 3.5 MHz (3,500,000 T-states/s)
Video	ULA: 256×192 paper, 15 colours; rendered with border at 320×240
Frame	50 Hz, 69,888 T-states/frame (312 lines × 224 T)
Audio	1-bit beeper (ULA bit 4); AY-3-8912 PSG on the HC-128
Memory	16K ROM + 48K RAM (HC-91); 128K banked + AY (HC-128); disk + CP/M (HC-2000)
I/O	ULA 0xFE; Kempston 0x1F; AY 0xFFFD/0xBFFF; i8272 0x85/0x87

The HC-91 ROM differs from the Sinclair 48K ROM by only ~50 bytes (copyright banner + a small CP/M boot stub), so the machine is fully 48K-Spectrum software compatible—a fact the project leans on hard.

2 Repository layout

Path	Role
<code>src/z80.[ch]</code>	bus-callback Z80 core (no machine knowledge)
<code>src/machine.[ch]</code>	memory map, I/O bus, frame loop, file dispatch
<code>src/video.c</code>	beam-accurate renderer + ROM-font OCR
<code>src/wav.c</code>	beeper → 44.1 kHz WAV synthesis
<code>src/ay.c</code>	AY-3-8912 PSG (HC-128)
<code>src/tape.c</code>	<code>.tap/.tzz/.wav</code> : trap + pulse loader + SAVE
<code>src/fdc.[ch]</code>	uPD765/i8272 floppy controller (HC-2000)
<code>src/snapshot.c</code>	<code>.sna/.z80/.szx/.scr</code> load + save
<code>src/rzx.[ch]</code>	RZX input record/replay (deterministic regression)
<code>src/keys.c</code>	scheduled keyboard events, typing, joysticks
<code>src/disasm.[ch]</code>	full-coverage Z80 disassembler
<code>src/debug.[ch]</code>	scriptable monitor: <code>bp/watch/step/trace</code>
<code>src/inflate.[ch]</code>	built-in DEFLATE/zlib inflater (SZX/RZX)
<code>src/png.c</code>	minimal PNG writer (no deps)
<code>src/sdl.c</code>	optional SDL2 frontend (<code>dlopen</code> 'd at runtime)
<code>src/main.c</code>	CLI driver: option parse, run loop, output
<code>src/zexrun.c</code>	standalone CP/M harness for <code>zexdoc/zexall</code>
<code>tests/</code>	unit + integration tests, golden images, test-tap generators
<code>tools/</code>	ROM/library fetchers, deb/Windows packaging
<code>.github/workflows/ci.yml</code>	download-free CI (build + ASan/UBSan)

The whole emulator is ~7,700 lines of C99 with **zero external dependencies** (`libc` + `-ldl` for the optional SDL runtime binding). PNG encoding, DEFLATE inflation, WAV writing and the SDL2 ABI slice are all implemented in-tree.

3 Layered architecture (Block Definition Diagram)

The design is a clean, dependency-pointing-downwards stack. The Z80 core sits at the bottom and knows nothing about the machine; everything above plugs bus behaviour into it through callbacks.

The single shared state object is `struct Machine` (`machine.h:114`), an aggregate owning the CPU, all memory banks, every peripheral and the framebuffer. It is allocated statically in `main()` (off the stack, because it is large) and passed by pointer everywhere. There is **no global mutable**

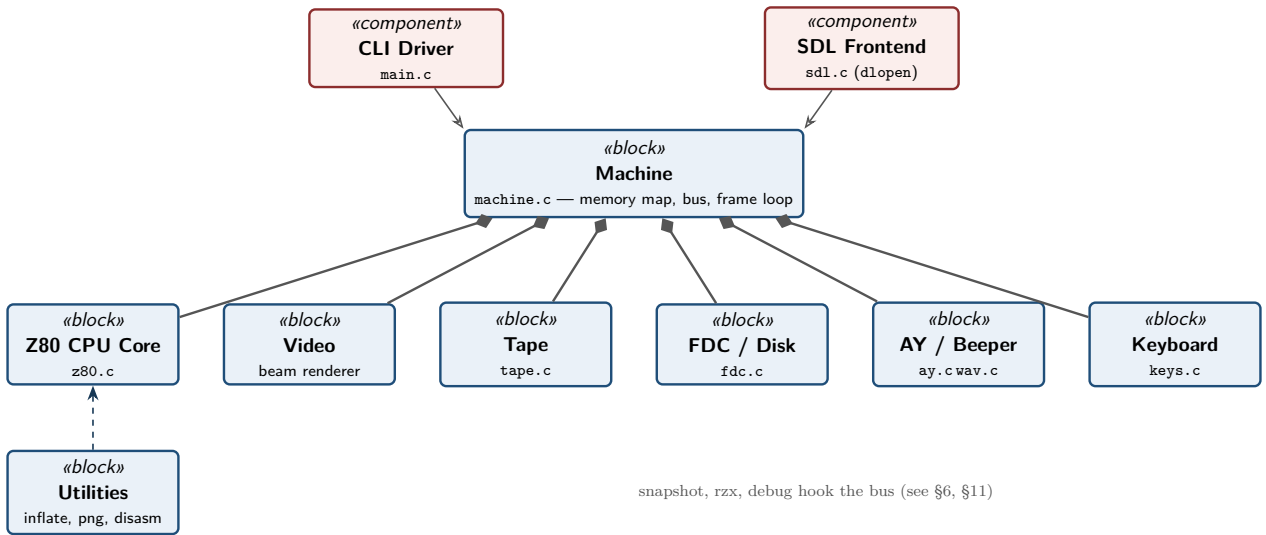


Figure 1: Block Definition Diagram: `hc91emu` is composed of a bus-agnostic Z80 core, owned by a single `Machine` block that also owns every peripheral and the renderer. Filled diamonds (◆) denote composition; plain arrows denote usage.

`state` outside it—which is what makes the test harness (`zexrun.c`) and the headless/SDL split trivial.

4 Machine internal structure (Internal Block Diagram)

The IBD below shows how the Z80 core connects to the emulated hardware through *six bus-callback ports*, and how the bus fans out to memory and the peripheral decoders.

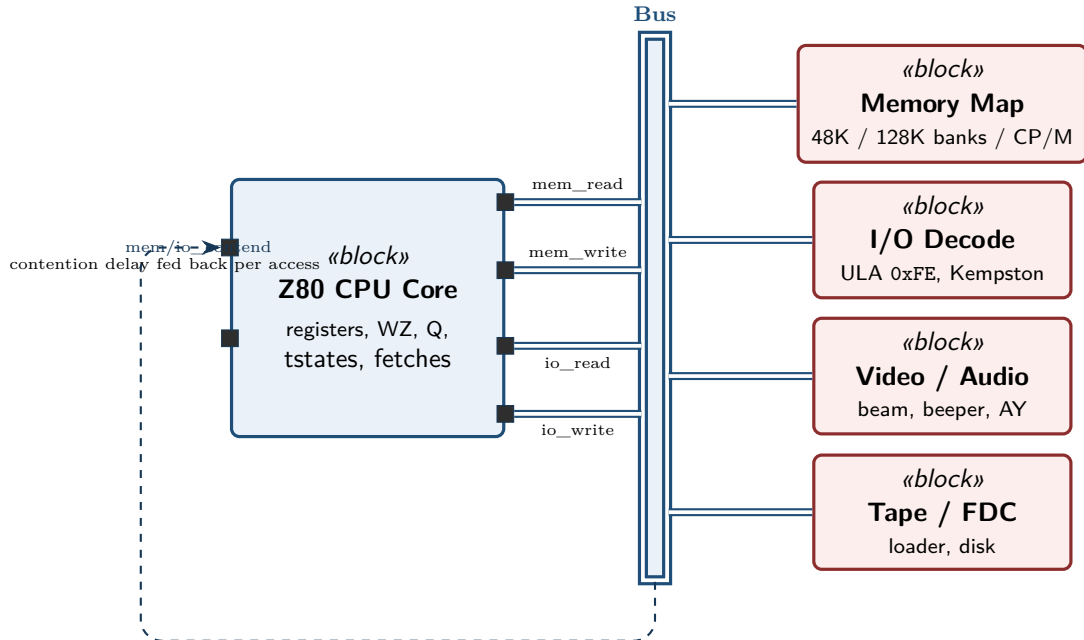


Figure 2: Internal Block Diagram: the Z80 core sees only six callback ports. The `Machine` block binds them to the bus, which decodes memory and the peripheral ports. Contention is fed *back* into the core per access.

The core never touches memory directly. The two wrappers `bus_io_read` and `bus_io_write` (`machine.c:251,266`) splice in **RZX playback/recording** and **debugger watchpoints** on top of the raw decode, without polluting the timing-critical path.

5 The Z80 CPU core (src/z80.c)

This is the heart of the project (~1,140 lines) and where emulation accuracy lives. It is deliberately **bus-agnostic** (see the IBD above).

5.1 Register model

`struct Z80` (`z80.h:18`) holds the full documented set plus the hardware-internal state needed for cycle-exact behaviour: main + alternate register pairs via a little-endian `RegPair` union, `ix/iy/sp/pc`, the internal **WZ** (`memptr`), the internal **Q** register (previous F if the last instruction modified flags), `iff1/iff2`, `im`, `halted`, `ei_pending`, and two 64-bit clocks: `tstates` (master) and `fetches` (every R bump; RZX currency).

5.2 Incremental T-state accounting

Timing is charged **per M-cycle**, not per instruction. The load-bearing helpers (`z80.c:58-108`):

- `ct(addr, len)` — add contention for `addr`, then `len` T;
- `intern(addr, n)` — `n` individually-contended 1T cycles;
- `mrd/mwr` — 3T memory access (contention first);
- `fetch_op` — 4T M1 fetch, bumps R and `fetches`;
- `io_in/io_out` — full 4T I/O: early contention, 1T, late contention, value transfer, 3T.

Because contention is evaluated *at the address and time of each access* (the callee sees an up-to-date `z->tstates`), the CPU timeline tracks real hardware exactly—and the floating bus + beam video fall out for free.

5.3 Instruction decode

`z80_step()` (`z80.c:1050`) consumes any run of `0xDD/0xFD` prefixes (only the last wins, as on real hardware) into a `pfx` selector, then dispatches on `0xCB/0xED/main`. `q` is set from a precomputed `*_modifies_f()` predicate (`z80.c:1014`)—notably `POP AF` and `EX AF, AF'` do **not** latch Q, matching Zilog NMOS (a real bug caught by Rak's `z80ccf`). Interrupts: `z80_int(bus)` (`z80.c:1092`) implements the 7T INT ack; `z80_nmi()` the 5T NMI to `0x0066`.

Validation: `zexdoc/zexall 67/67`; Rak's `z80full/ccf/memptr 160/160`; all 162,000 `SingleStepTests/z80` vectors (replayed by `tests/sst.c`).

6 The Machine layer (src/machine.c)

6.1 Memory maps

`machine_peek()` (`machine.c:22`) and `bus_mem_write()` (`machine.c:69`) encode the map, branching on the active model:

- **48K (HC-85/90/91, plain 48K):** `mem[65536]` with low 16K ROM write-protected. `mem_cpm[0x4000]` is a separate low-RAM bank paged over ROM by **port 0x7E bit 0**—the HC-91's signature CP/M feature (the ROM bootstrap at `0x386E` does `OUT (0x7E), 1; JP 0`). It is pre-filled with `0x76` (HALT).
- **128K (HC-128):** eight 16K banks in `ram128[8]`: `0x4000=bank5`, `0x8000=bank2`, `0xC000=bank` (latch&7); `port_7ffd` bits select shadow screen (3), ROM (4), lock (5); odd banks contended. `m->screen` always points at the active display bank so the renderer is model-agnostic.
- **HC-2000:** the most complex map. The `cfg_7e` system latch (`(port&0x81)==0`, read-back, self-locking) selects BASIC vs CP/M ROM, can relocate the ROM window to `0xE000` with RAM low, and relocate video to `0xC000`; a separate CPM flip-flop (write `0xC7` set / `0xC5` clear) pulls

A13 high so 0xE000 RAM appears at 0xC000. Plus the "IF1" disk interface: an 8K shadow ROM (rom_if1) paged at 0x0008/0x1708 and out after 0x0700, with its own 16K interface RAM.

6.2 Contention model

contention_delay() (machine.c:126) implements the canonical ULA stall pattern 6,5,4,3,2,1,0,0 per 8 T-states, starting at **T=14335** after the frame interrupt, for the first 128 T of each of the 192 display lines. bus_io_contend_late (machine.c:168) re-evaluates contention at each simulated sub-cycle for non-ULA contended ports.

6.3 I/O bus

bus_io_read_raw() (machine.c:187) decodes, in order: the HC-2000 config latch, the ULA (even addresses: keyboard matrix + EAR—which carries the tape signal while playing, else follows the speaker bit, Issue-3), Kempston (0x1F), AY read (0xFFFF), the FDC (0x85/0x87). Anything unattached falls through to the **floating bus** (machine.c:231): while the ULA fetches display data it leaves that byte on the bus—essential for games like Arkanoid that beam-sync on it.

7 The frame loop (Activity Diagram)

machine_run_frame() (machine.c:501) is the master clock driver. The activity diagram below is a faithful transcription.

8 Beam-accurate video (src/video.c)

Rather than rendering once per frame, the display is **painted incrementally at the beam position**, so mid-frame effects are correct.

9 Tape subsystem (src/tape.c)

Two loading paths share one pulse stream:

- **Trap loading** (default, instant): when PC hits 0x0556 (LD-BYTES), tape_trap() copies the next block straight into RAM.
- **Pulse loading** (--real-tape): the whole tape is compiled at load time into a flat list of pulse durations (tape.c:33) using standard ROM timings; EAR toggles per edge against the cycle-exact clock (tape_player_ear, tape.c:700).

Formats: .tap (raw), .ttx (turbo, CSW 0x18 RLE+zlib, generalized-data 0x19), sampled .wav (DC removal + Schmitt trigger, 4–192 kHz). SAVE is trapped at 0x04C2 (SA-BYTES) and captured byte-exact to .tap.

9.1 Tape player state machine

The player's pause/resume logic at block boundaries is what makes multi-load schemes ("stop the tape / press any key", rewind prompts) work without a UI. The figure below models that behaviour.

10 Disk & CP/M (src/fdc.c) — HC-2000

A polled uPD765/i8272 emulation: FDC at 0x85/0x87 with a control latch at 0x05/0x07 (TC, drive select, reset). Implemented commands: SPECIFY, SENSE DRIVE/INTERRUPT STATUS, RECALIBRATE, SEEK, READ ID, READ/WRITE DATA (multi-sector, TC/EOT terminated), FORMAT TRACK; unknown commands return the invalid ST0. Seeks are instantaneous; transfers

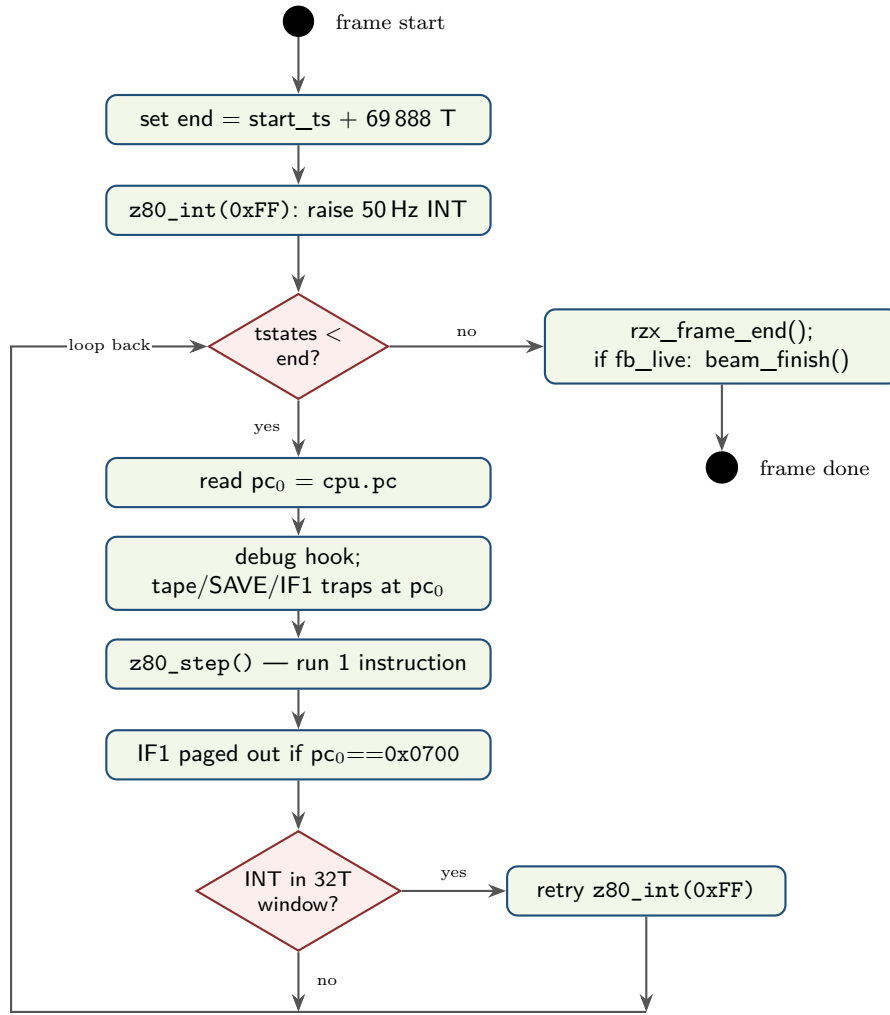


Figure 3: Activity Diagram of one emulated frame. Three properties matter: **(1) drift-free**—frame boundaries are exact multiples of 69,888 T (overshoot carries forward); **(2)** the INT window is the real ~ 32 T; **(3)** ROM traps and IF1 paging are PC predicates evaluated before each step.

are byte-pollled through MSR RQM/DIO. Raw `.img/.dsk` images in the four period geometries (640K/720K 80-track, 320K/360K 40-track) are recognised by size; dirty images write back at exit. Together with the HC-2000 memory map (§6) this boots **CP/M 2.2** from the system tracks to the **A>** prompt with working DIR.

11 State: snapshots & RZX

Snapshots (`src/snapshot.c`): read/write for `.sna` (48K), `.z80 v1/v2/v3` (48K and 128K mode 3, pages 3–10; v1 RLE), `.szx` (zx-state v1.4, with **zlib-compressed RAM pages** loaded through the built-in `inflate.c`), raw `.scr`. HALT state survives saving by re-pointing PC at the HALT opcode (`snapshot.c:56`).

RZX (`src/rzx.c`) is a record/replay pair for deterministic whole-session regression. The sequence diagram below shows the replay contract.

12 Input, debugger, frontend, CLI

Input (`src/keys.c`): `keys_apply(frame)` materialises the 8-row keyboard matrix from `KeyEvent` ranges each frame. Three input modes build them: `keys_type` (types a BASIC string with keyword/symbol composition), `keys_raw` (a raw chord by name), `keys_joy` (joystick routed by

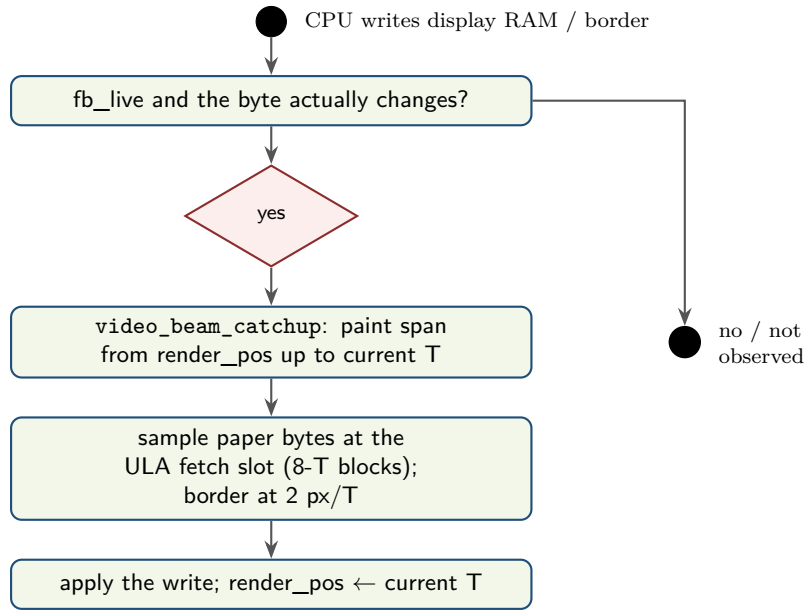


Figure 4: Activity Diagram of the lazy catch-up painter. Painting is gated on `fb_live`: turbo/headless runs that nobody observes skip it entirely (`main.c` sets `fb_live=(f==frames-1)`). `paint_cell()` also models **ULA snow**: with `I` in `0x40-0x7F` the low 7 bits of the fetch address come from `R`. `video_screen_text()` does **ROM-font OCR** (match each 8×8 cell against the font at `0x3D00`)—how the test suite asserts on screen contents without image diffing.

--joy-type to Kempston/Sinclair/cursor). Kempston is gated behind `--kempston` because an *absent* interface must leave `0x1F` floating.

Disassembler (`src/disasm.c`): full-coverage, decoded algorithmically by the `x/y/z/p/q` bit fields rather than a 256-entry table, so all prefixes, undocumented result-copy forms, `ED` no-ops and dead prefixes are uniform. Locked by `tests/dtest.c` (124 cases).

Monitor (`src/debug.c`): PC breakpoints, memory read/write and I/O-port watchpoints (each ≤ 32), single-step, hex dump, registers with frame-relative T-state, per-instruction trace-to-file. Scriptable via `--debug "regs;step 3;cont"` then `stdin` (auto-continues when both are exhausted so headless runs never hang). The `debug_note_*` hooks arm a stop that fires before the next instruction—clean separation from the timing-critical core.

SDL frontend (`src/sdl.c`): the standout choice—SDL2 is `dlopen`’d at runtime and a minimal ABI slice declared in-tree, so the build needs no SDL headers or link libraries. Features: resizable window, **audio-clocked 50 Hz pacing** (20 ms tick fallback), full host-keyboard→matrix map, first game controller → Kempston, turbo/pause/quit, quick save/load, fullscreen, drag-and-drop file loading.

CLI (`src/main.c`): ~520-line option parser/orchestrator. ROM path resolution (`main.c:74`) prefers the relative path (in-tree) and falls back to `HC91_DATADIR (/usr/share/hc91emu)`.

13 Testing & validation

Layer	What	How
CPU unit	T-state totals, contention M-cycles, disassembly	<code>ttest/ctest/dtest</code> — pure, no ROMs
CPU exerciser	documented + undocumented flags	<code>zexrun</code> runs <code>zexdoc/zexall.com</code>
CPU vectors	full per-instruction bus traces	<code>sst</code> replays 162 SingleStepTests files (162,000)
In-machine	Rak’s <code>z80full/ccf/memptr</code>	loaded as tapes; screen asserts “all tests passed”
Integration	boot, BASIC, games, audio, tape, disk, CP/M, SDL	golden PNGs + OCR + WAV pitch

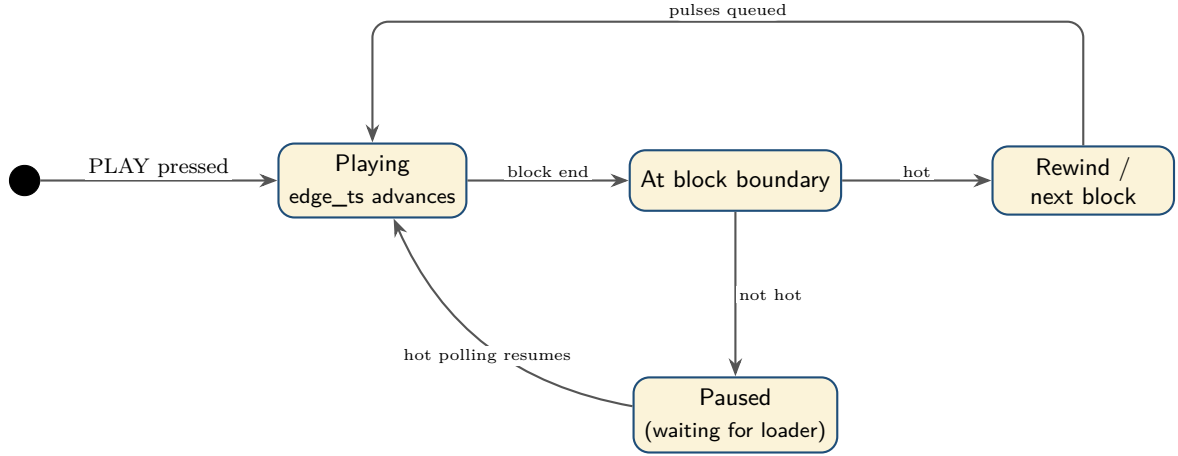


Figure 5: State-Machine Diagram of the pulse player. *Hot* = ≥ 32 EAR reads per 10,000-T window, excluding the ROM keyboard scanner (0x028E-0x02BE). `--tape-b + tape_swap()` model a second cassette side.

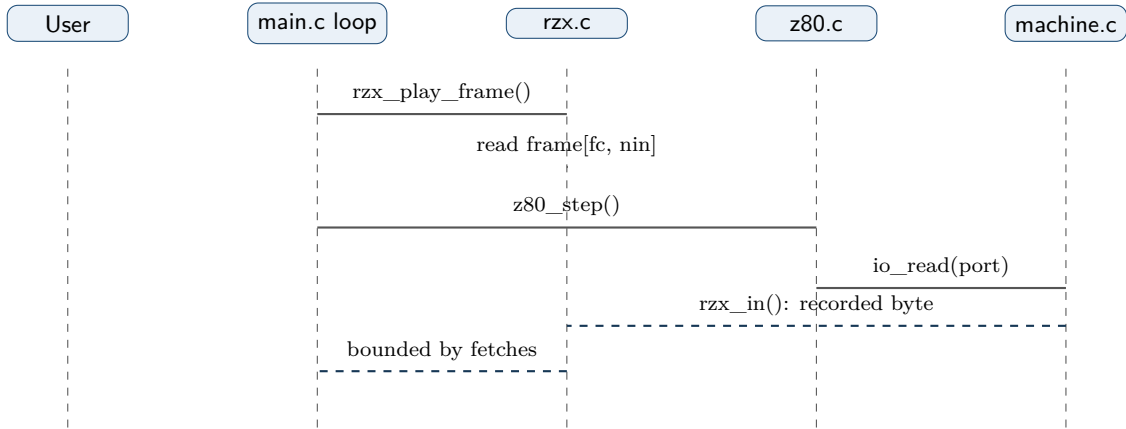


Figure 6: Sequence Diagram: during RZX playback, port reads are fed from the recording. Frames are bounded by the `fetches` counter (bumped on every R increment), so a captured session replays bit-exactly. Recording embeds a `.z80` snapshot at start.

Key ideas: generated `.tap` files (`bordertap.c`, `multitap.c`) produce *known* beam effects to lock the renderer; `--text` OCR avoids image-diff fragility; `--fb-dump + fbcheck.c` do pixel-exact checks; RZX gives bit-exact whole-session regression. **CI** runs only the download-free subset (no ROMs, no copyrighted data, no network): `build + ttest/ctest/dtest/zexdoc`, repeated under **ASan** + **UBSan**.

14 Notable engineering decisions

1. **Bus-callback CPU core** (`z80.c`)—the single most important abstraction: makes the core reusable (`zexrun`), testable in isolation, and decouples timing accuracy from machine specifics.
2. **Incremental per-M-cycle T-state accounting** with contention evaluated at access time—the foundation making contention, floating bus and beam video all cycle-exact simultaneously.
3. **Zero external dependencies**—a hand-written DEFLATE inflater, PNG encoder and SDL ABI slice keep the build trivial and the binary self-contained.
4. **dlopen SDL** instead of a build-time dependency—headless stays the default, tests need no display, one `make` target builds all.

5. **One aggregate Machine struct, no globals**—clean ownership, easy to reset/snapshot/reason about.
 6. **Drift-free frame loop** (exact 69,888-T boundaries, overshoot carried forward) keeps phase locked indefinitely.
 7. **Test-tape generation + OCR assertion** instead of brittle image diffs—deterministic and human-readable.
 8. **Going beyond MAME:** MAME’s `hc91` driver is a plain ROM swap; this emulator implements the actual CP/M paging, the HC-128’s genuine banking+AY (confirmed by disassembling its ROM), and the HC-2000’s full disk interface + CP/M boot.
-

Source under the MIT License; the grant covers the emulator only—ROMs and period software are never committed and are fetched on demand by `tools/get_roms.sh`. Derived from a source audit of the `cosmindxu/hc91emu` tree (~7,700 lines of C99).